

AI Lab assignment 12
Tejas Patil
U24AI061

Q1)

```
domains = {
    "P1": ["R1", "R2", "R3"],
    "P2": ["R1", "R2", "R3"],
    "P3": ["R1", "R2", "R3"],
    "P4": ["R1", "R2", "R3"],
    "P5": ["R1", "R2", "R3"],
    "P6": ["R1", "R2", "R3"]
}

constraints = {
    "P1": ["P2", "P3", "P6"],
    "P2": ["P1", "P3", "P4"],
    "P3": ["P1", "P2", "P5"],
    "P4": ["P2", "P6"],
    "P5": ["P3", "P6"],
    "P6": ["P1", "P4", "P5"]
}

def revise(i, j):
    revised = False
    before = domains[i][:]    # store old domain

    for value in domains[i][:]:
        supported = False
        for other in domains[j]:
            if value != other:
                supported = True
                break

    if not supported:
        domains[i].remove(value)
        revised = True
```

```

        return revised, before, domains[i]

def ac_3():
    q = []
    for i in constraints:
        for j in constraints[i]:
            q.append((i,j))

    count = 0

    while len(q) != 0:
        count += 1
        i, j = q.pop(0)

        change, before, after = revise(i, j)

        if count <= 5:
            print(f"\nChecking {i} -> {j}")
            print(f"Before: {i} = {before}")

            if change:
                print(f"After : {i} = {after}    (Reduced)")
            else:
                print(f"After : {i} = {after}    (No Change)")

            if change:
                if len(domains[i]) == 0:
                    return False

                for k in constraints[i]:
                    if k != j:
                        q.append((k, i))

    return True

domains["P1"] = ["R1"]

if ac_3():

```

```
print("\nFinal Result:")
print("Arc consistent")
print(domains)
else:
    print("Failure")
```

Q2)

```
grid = [
    [0,0,0,0,0,6,0,0,0],
    [0,5,9,0,0,0,0,0,8],
    [2,0,0,0,0,8,0,0,0],
    [0,4,5,0,0,0,0,0,0],
    [0,0,3,0,0,0,0,0,0],
    [0,0,6,0,0,3,0,5,0],
    [0,0,0,0,0,7,0,0,0],
    [0,0,0,0,0,0,0,0,0],
    [0,0,0,0,5,0,0,0,2]
]

domains = {}
for i in range(9):
    for j in range(9):

        if grid[i][j]==0:
            domains[(i,j)]=[1,2,3,4,5,6,7,8,9]
        else:
            domains[(i,j)]=[grid[i][j]]

constraints = {}
for i in range(9):
    for j in range(9):
        constraints[(i,j)]=[]
        for col in range(9):
            if col!=j:
                constraints[(i,j)].append((i,col))
```

```

        for row in range(9):
            if row!=i:
                constraints[(i,j)].append((row,j))

        box_row = (i//3)*3
        box_col = (j//3)*3

        for r in range(box_row,box_row+3):
            for c in range(box_col,box_col+3):

                if (r,c)!=(i,j) and (r,c) not in constraints[(i,j)]:
                    constraints[(i,j)].append((r,c))

def revise(i,j):
    revised=False
    for value in domains[i][:]:
        supported=False
        for other in domains[j]:
            if value!=other:
                supported=True
                break
        if not supported:
            domains[i].remove(value)
            revised=True

    return revised

def ac_3():

    q=[]
    for i in constraints:
        for j in constraints[i]:
            q.append((i,j))

    while len(q)!=0:
        i,j=q.pop(0)

```

```

        change=revise(i,j)
    if change:
        if len(domains[i])==0:
            return False
        for k in constraints[i]:
            if k==j:
                continue
            q.append((k,i))

    return True

before=0
for i in domains:
    before += len(domains[i])

if ac_3():
    after=0
    for i in domains:
        after+=len(domains[i])
    removed=before-after
    print("Total Values Removed =",removed)
    print("\nDomain Size Grid:\n")

    for i in range(9):
        for j in range(9):
            print(len(domains[(i,j)]),end=" ")
        print()

    empty=False
    solved=True
    for i in domains:
        if len(domains[i])==0:
            empty=True

        if len(domains[i])!=1:
            solved=False
    print()

```

```
    if empty:
        print("Puzzle Unsolvable")
    elif solved:
        print("Puzzle Completely Solved")
    else:
        print("Puzzle Partially Reduced Only")
else:

    print("Failure")
```